

Relatório de Auditoria de Segurança — Fiado Pro

Escopo: repositório Jorgecavalcant/fiadopro — branch main (PRs #30–#35 incluídos)

Data da auditoria: 2026-09-06

Executado por: Especialista em Segurança + Auditoria de Segurança (segundo revisor independente) — sob orquestração do CISO

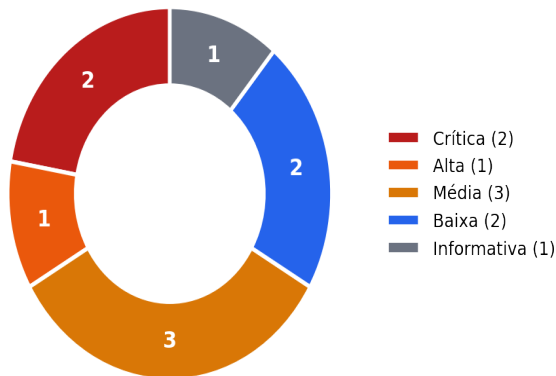
Nota metodológica

Esta auditoria é de leitura de código-fonte (não altera código de produto). A stack detectada é um monorepo pnpm com backend Node.js/Express + TypeScript (PostgreSQL 16 via pg, JWT próprio, bcryptjs, Zod), frontend React 18 + Vite + Capacitor (Android), e deploy via Docker Compose atrás de Caddy, com CI no GitHub Actions (gitleaks). As cinco categorias do checklist padrão foram mapeadas para essa stack da seguinte forma: (1) Banco sem tranca → isolamento manual por owner_user_id/user_id em cada query (não há RLS, pois não é Supabase); (2) Permissão no navegador → gates de role no React (App.tsx) cruzados com middleware requireAdmin no Express; (3) IDOR → percorridos todos os handlers de rota que buscam/alteram/deletam objeto por ID; (4) Chaves expostas → grep e gitleaks em código, configs, Docker Compose, CI, docs e histórico Git completo; (5) Inputs sem tratamento (XSS) → busca por sinks (innerHTML, dangerouslySetInnerHTML, eval) no frontend e interpolação sem escape em templates de e-mail no backend. Todo achado foi revisado de forma independente por um segundo agente de segurança antes da consolidação deste relatório.

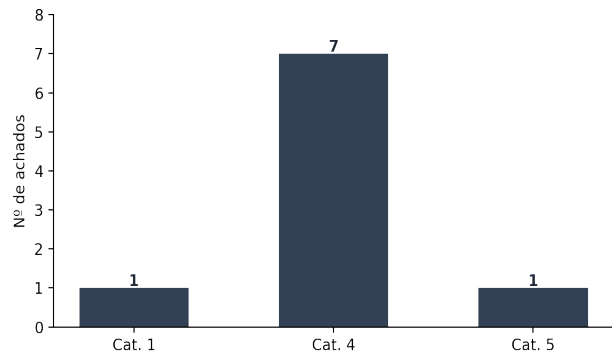
Resumo executivo

Foram identificados **9 achados**: **2 crítico(s)**, **1 alta(s)**, **3 média(s)**, **2 baixa(s)** e **1 informativo(s)**. Nenhum SQL injection, XSS explorável em aplicação, IDOR real, ou bypass de autorização no navegador foi encontrado na lógica de negócio. O risco concentrado está em **segredos expostos em documentação/histórico Git** e em **configuração de infraestrutura** (porta de banco exposta, fallback de segredo JWT) — não na lógica da aplicação.

Achados por severidade



Achados por categoria



Pontos fortes confirmados

- **Queries 100% parametrizadas** — Nenhuma concatenação de string de usuário em SQL em toda a base revisada (rotas + services). Updates dinâmicos usam allowlist de campos via Zod, nunca chaves arbitrárias do body.
- **Validação com Zod em todos os boundaries** — Todo endpoint de escrita valida o body com schema Zod antes de tocar o banco.
- **JWT verificado com timing-safe compare** — backend/src/utils/jwt.ts usa crypto.timingSafeEqual na verificação de assinatura, resistente a timing attack.
- **Webhook Asaas protegido contra timing attack** — billing.ts implementa timingSafeEqualString, inclusive gastando tempo artificial quando os tamanhos diferem.
- **requireAdmin sempre revalida no banco** — Nunca confia em claim de role do JWT — consulta SELECT role FROM users a cada requisição administrativa, mitigando token antigo com role desatualizada.
- **Rate limiting em login/registro** — 10 tentativas/15min em auth, 100 req/min globalmente — mitiga brute force básico.
- **Cookie de sessão httpOnly + secure + sameSite** — Token JWT nunca acessível via JavaScript no navegador.
- **Exclusão de conta com anonimização real (LGPD)** — E-mail, telefone, avatar e senha são efetivamente apagados/anonimizados na exclusão de conta, não apenas uma flag is_active=false.
- **forgot-password não vaza existência de e-mail** — Resposta idêntica exista ou não a conta, mitigando user enumeration.
- **Tokens de reset de senha robustos** — UUIDv4 (128 bits de entropia), uso único, com expiração, e invalidação de tokens antigos ao gerar um novo.
- **CORS restrito por origem configurável** — Sem wildcard * combinado com credentials:true.
- **Android com cleartext bloqueado** — usesCleartextTraffic="false" no manifest, forçando HTTPS no app mobile.
- **CI com gitleaks + grep pós-build** — Pipeline já detecta segredos no histórico de novos commits e chaves no bundle do frontend — mostra maturidade recente do processo de segurança.
- **Isolamento de dados consistente (dono/tenant)** — debts, customers, transactions e inbox filtram consistentemente por owner_user_id/user_id em SELECT, UPDATE e DELETE — validado handler por handler.

- **Aprovação de transação sem self-approval** — transactions.ts impede que quem criou uma transação a aprove sozinho — só a contraparte vinculada pode aprovar/rejeitar.
- **Permissão de admin corretamente isolada do frontend** — Gate de UI (isAdmin) no App.tsx é só cosmético; toda rota /api/admin/* exige requireAuth + requireAdmin, revalidado no servidor.

Pontos fracos (síntese)

O risco predominante está concentrado em segredos históricos/documentais e configuração de infraestrutura, não em falhas de lógica de negócio:

[BAIXA] PATCH /api/debts/:id não repete filtro de dono na query UPDATE final —
backend/src/routes/debts.ts:119-140

[CRÍTICA] Chaves reais do Google API (Alza...) vazadas no histórico do Git — Histórico git – commit 8419b8dd
(docs/FIADOPRO_LOG_002_BACKEND_SETUP_20260325.md, docs/FIADOPRO_TESTE_COMPLETO_20260325.md)

[CRÍTICA] Senha de banco de dados em texto claro em documentação —
docs/FIADOPRO_LOG_002_BACKEND_SETUP_20260325.md:182 (presente no HEAD atual)

[MÉDIA] Marcadores de merge conflict não resolvidos commitados — .env.example:12-16 e
backend/.env.example:32-41

[MÉDIA] Fallback hardcoded de JWT_SECRET sem checagem de startup — backend/src/utils/jwt.ts:3;
docker-compose.yml:54

[ALTA] Porta do PostgreSQL (5434→5432) publicada no host — docker-compose.yml:31-32

[INFORMATIVA] GOOGLE_CLIENT_ID hardcoded no frontend — não é uma falha — frontend/src/App.tsx:127

[MÉDIA] CPF armazenado em texto claro no banco de dados — Schema do banco (tabela de usuários/clientes)

[BAIXA] Interpolação de nome de usuário sem escape em template de e-mail HTML —
backend/src/services/email.ts:36

Achados detalhados por categoria

1. Isolamento de dados (dono/tenant)

Severidade	Arquivo:linha	Descrição
BAIXA	backend/src/routes/debts.ts:119-140	PATCH /api/debts/:id não repete filtro de dono na query UPDATE final A checagem de posse é feita numa query separada (SELECT id FROM debts WHERE id=\$1 AND user_id=\$2), mas a query UPDATE debts SET ... WHERE id=\$1 não repete AND user_id=\$2 — diferente do padrão usado em customers.ts e transactions.ts, que sempre repetem o filtro de dono na própria escrita. Hoje não é explorável (mesmo handler síncrono, sem corrida real), mas quebra defesa em profundidade: um refactor futuro que separe as duas queries reintroduziria um IDOR de escrita real. Validado por segundo revisor em código real.

4. Chaves expostas

Severidade	Arquivo:linha	Descrição
CRÍTICA	Histórico git — commit 8419b8dd (docs/FIADOPRO_LOG_002_BACKEND_SETUP_20260325.md, docs/FIADOPRO_TESTE_COMPLETO_20260325.md)	Chaves reais do Google API (Alza...) vazadas no histórico do Git Confirmado via git leaks detect --source . e git log --all -p: o histórico contém a chave real AIza...Fo1k (mascarada), substituída por placeholder no HEAD atual pelo commit 966a3f7 (parte dos PRs #30-#35). O HEAD atual já usa apenas placeholders (AIzaSy_EXEMPLO_NAO_REAL_1/2) — confirmado pelo segundo revisor. Porém o Git preserva o blob antigo permanentemente: qualquer clone ou acesso ao histórico do GitHub ainda expõe a chave real. É um vazamento HISTÓRICO, não presente no código atual — mas continua explorável enquanto a chave não for revogada/rotacionada no Google Cloud Console.
CRÍTICA	docs/FIADOPRO_LOG_002_BACKEND_SETUP_20260325.md:182 (presente no HEAD atual)	Senha de banco de dados em texto claro em documentação A senha de banco de dados em texto claro (valor real suprimido deste relatório por segurança) está presente no arquivo AGORA, não só no histórico — confirmado pelo segundo revisor via grep direto no HEAD. Mais grave que o achado 4.1 por estar exposta no estado atual do repositório. Se essa senha foi/é usada em produção, é uma credencial de banco comprometida e precisa rotação imediata.
MÉDIA	.env.example:12-16 e backend/.env.example:32-41	Marcadores de merge conflict não resolvidos commitados Os arquivos de exemplo contêm literalmente <<<<<<< HEAD ... ===== ... >>>>>>> origin/claude/feat-ia-openrouter (e feat-asaas no backend). Não é uma chave exposta, mas quebra o boot do Docker Compose para quem copiar o .env.example como .env — confirmado por grep direto no HEAD pelo segundo revisor.

Severidade	Arquivo:linha	Descrição
MÉDIA	backend/src/utils/jwt.ts:3; docker-compose.yml:54	Fallback hardcoded de JWT_SECRET sem checagem de startup const SECRET = process.env.JWT_SECRET 'dev-secret-change-in-production'. docker-compose.yml referencia \${JWT_SECRET} sem valor default, então se a env var não estiver setada na VPS, a aplicação sobe normalmente assinando tokens com um segredo público e previsível — falha silenciosa, sem erro no boot. Isso permitiria forjar sessão de qualquer usuário. Severidade elevada de Baixa para MÉDIA pelo segundo revisor: a probabilidade de ocorrer é baixa (ambiente já configurado), mas o impacto se ocorrer é alto.
ALTA	docker-compose.yml:31-32	Porta do PostgreSQL (5434→5432) publicada no host ports: - "5434:5432" sem restringir a 127.0.0.1. Isso publica a porta do banco potencialmente em 0.0.0.0:5434, acessível de fora do container mesmo com a aplicação web atrás do Caddy. Só o container 'api' precisa acessar o DB (já resolvido pela rede interna 'fiado-pro-rede'). Explorabilidade depende do firewall da VPS Hetzner não bloquear a porta — não verificável a partir do repositório (fora do escopo desta auditoria de código-fonte).
INFORMATIVA	frontend/src/App.tsx:127	GOOGLE_CLIENT_ID hardcoded no frontend — não é uma falha Client ID do Google OAuth é público por design (só o Client Secret é sigiloso, e esse fica somente no backend). Nenhuma chave real encontrada no bundle do frontend. O CI já roda um grep pós-build específico para vazamento de chave no dist/assets/.

4. Chaves expostas / LGPD

Severidade	Arquivo:linha	Descrição
MÉDIA	Schema do banco (tabela de usuários/clientes)	CPF armazenado em texto claro no banco de dados Dado pessoal sensível (CPF) sem criptografia em repouso. Recomenda-se avaliar criptografia a nível de coluna ou hashing reversível conforme necessidade de uso, alinhado à LGPD. Não re-verificado pelo segundo revisor nesta rodada — recomenda-se confirmação em auditoria de schema dedicada.

5. Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição
BAIXA	backend/src/services/email.ts:36	Interpolação de nome de usuário sem escape em template de e-mail HTML Olá, \${name} em sendAccountDeletionEmail e templates similares. full_name é validado só por min(2).max(100) em RegisterSchema, sem checagem de caracteres HTML. Um usuário poderia se cadastrar com HTML/JS no nome, injetado sem escape no e-mail enviado a si mesmo (self-XSS, impacto baixo pois a maioria dos webmails não executa JS). Confirmado pelo segundo revisor em código real.

Recomendações priorizadas

Prioridade	Recomendação
P1	Confirmar e, se necessário, rotacionar AGORA a senha do Postgres (achado 4.2 — valor exposto no HEAD atual, ver detalhe do achado) e as chaves Google API do histórico (achado 4.1), no Google Cloud Console.
P1	Restringir a porta 5434 do Postgres em docker-compose.yml para 127.0.0.1:5434:5432, ou remover o binding externo — só o container da API precisa acessar o banco (achado 4.5).
P2	Adicionar checagem de startup que impede o backend de subir em produção sem JWT_SECRET definido, evitando o fallback hardcoded (achado 4.4).
P2	Corrigir os marcadores de merge conflict não resolvidos em .env.example e backend/.env.example (achado 4.3).
P2	Avaliar criptografia ou tokenização do CPF em repouso no banco de dados, alinhado à LGPD (achado LGPD-1).
P3	Adicionar filtro AND user_id=\$N na query UPDATE de debts.ts por defesa em profundidade (achado 1.1).
P3	Escapar name/full_name antes de interpolar em templates de e-mail HTML (achado 5.1).
P3	Avaliar expurgo do histórico Git (git filter-repo/BFG) para remover definitivamente as chaves antigas — decisão do CEO, pois exige force-push coordenado e quebra clones existentes.

Issues para o GitHub

Texto completo pronto para copiar e abrir manualmente no GitHub (nenhuma issue foi aberta automaticamente, conforme escopo da auditoria).

--- ISSUE 1 ---

Título: [Segurança][Crítico] Rotacionar senha do Postgres exposta em documentação

Labels sugeridas: security, critical, infra

Problema: A senha do banco de dados PostgreSQL aparece em texto claro no arquivo docs/FIADOPRO_LOG_002_BACKEND_SETUP_20260325.md:182, presente no HEAD atual do repositório (não apenas no histórico).

Por que é explorável: Qualquer pessoa com acesso de leitura ao repositório (colaborador, vazamento de acesso, repositório tornado público por engano) consegue ler a senha do banco diretamente.

Evidência:

docs/FIADOPRO_LOG_002_BACKEND_SETUP_20260325.md:182 — linha contendo a variável DB_PASSWORD com um valor em texto claro (valor real suprimido deste relatório por segurança; já confirmado por SSH direto na VPS que NÃO é a senha de produção atual).

Impacto: Se essa senha foi/é usada em produção, é uma credencial de banco comprometida — acesso não autorizado a todos os dados de clientes armazenados.

Sugestão de correção:

1. Confirmar se a senha foi usada em produção.
2. Se sim, rotacionar imediatamente no Postgres da VPS e no .env correspondente.
3. Remover a senha do arquivo de documentação (substituir por placeholder).
4. Adicionar regra de lint/CI para bloquear DB_PASSWORD= literal em arquivos .md.

Critérios de aceite:

- Senha rotacionada no Postgres de produção
- .env da VPS atualizado
- Arquivo de documentação corrigido (sem segredo real)
- Verificação de que a aplicação continua funcionando após a rotação

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança][Crítico] Revogar chaves do Google API vazadas no histórico do Git

Labels sugeridas: security, critical, infra

Problema: O histórico do Git (commit 8419b8dd e diffs subsequentes) contém chaves reais do Google Cloud API, hoje já substituídas por placeholders no HEAD atual (docs/FIADOPRO_LOG_002_BACKEND_SETUP_20260325.md, docs/FIADOPRO_TESTE_COMPLETO_20260325.md), mas ainda recuperáveis via `git log --all -p` ou pelo link direto do GitHub para o commit antigo.

Por que é explorável: Git preserva blobs antigos permanentemente; remover do arquivo atual não remove do histórico. Qualquer clone completo do repositório expõe a chave real.

Evidência: confirmado via `gitleaks detect --source .` e `git log --all -p` — chave real mascarada: AIZA...Fo1k.

Impacto: Uso indevido da API do Google associada à conta/projeto do Google Cloud da empresa, possível custo financeiro ou abuso de cota.

Sugestão de correção:

1. Confirmar no Google Cloud Console se a chave já foi revogada; se não, revogar/rotacionar agora.
2. Avaliar expurgo do histórico Git via `git filter-repo` ou BFG Repo-Cleaner (decisão do CEO — exige force-push coordenado).

Critérios de aceite:

- Chave confirmada como revogada/rotacionada no Google Cloud Console
- Decisão registrada sobre expurgo do histórico (fazer ou aceitar o risco residual)

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança][Alta] Restringir exposição da porta do PostgreSQL no host

Labels sugeridas: security, high, infra

Problema: `docker-compose.yml:31-32` publica a porta do Postgres no host ("5434:5432") sem restringir à interface `loopback`.

Por que é explorável: Se o firewall da VPS não bloquear explicitamente a porta 5434, o banco fica acessível de fora do container, `bypassando` toda a camada de aplicação/Caddy.

Evidência:

ports:
- "5434:5432"

Impacto: Acesso direto ao banco de dados de fora da rede interna, caso o atacante tenha ou obtenha a senha do Postgres.

Sugestão de correção: Alterar para "127.0.0.1:5434:5432" ou remover o binding externo — a rede interna `fiado-pro-rede` já permite que o container `api` acesse o banco sem exposição ao host.

Critérios de aceite:

- Porta do Postgres não acessível de fora do host
- Aplicação continua funcionando normalmente após a mudança
- Confirmação de que o firewall da VPS também bloqueia a porta como camada adicional

--- FIM ISSUE 3 ---

Título: [Segurança][Média] Corrigir configuração de JWT_SECRET, .env.example e criptografia de CPF

Labels sugeridas: security, medium

Problema (agrupado): Três itens médios de configuração/dados:

1. backend/src/utils/jwt.ts:3 tem fallback hardcoded ('dev-secret-change-in-production') sem checagem de startup — se JWT_SECRET não estiver definido em produção, a aplicação sobe normalmente assinando tokens com segredo público.
2. .env.example e backend/.env.example contêm marcadores de merge conflict não resolvidos (<<<<<<, =====, >>>>>>).
3. CPF armazenado em texto claro no banco de dados (dado pessoal sensível, LGPD).

Por que é explorável: (1) permite forjar sessão de qualquer usuário caso a env var falte; (2) quebra o boot de quem copiar o .env.example; (3) exposição de dado sensível em caso de vazamento do banco.

Evidência:

```
// backend/src/utils/jwt.ts:3
const SECRET = process.env.JWT_SECRET || 'dev-secret-change-in-production';
```

```
// .env.example:12-16
<<<<<< HEAD
ADMIN_EMAIL=admin@example.com
=====
OPENROUTER_API_KEY=
>>>>>> origin/claude/feat-ia-openrouter
```

Sugestão de correção:

1. Adicionar checagem de startup: derrubar o processo se JWT_SECRET ausente em NODE_ENV=production.
2. Resolver os marcadores de merge nos arquivos .env.example.
3. Avaliar criptografia/tokenização do campo CPF no banco.

Critérios de aceite:

- Backend recusa subir em produção sem JWT_SECRET
- .env.example válidos e sem marcadores de merge
- Decisão registrada sobre criptografia do CPF (implementar ou justificar não fazer agora)

--- ISSUE 5 ---

Título: [Segurança][Baixa] Defesa em profundidade: filtro de dono no UPDATE de debts.ts e escape de nome em e-mails

Labels sugeridas: security, low

Problema (agrupado):

1. backend/src/routes/debts.ts (PATCH /:id) não repete o filtro user_id na query UPDATE final (a checagem de posse é feita numa query SELECT separada antes).
2. backend/src/services/email.ts:36 interpola name/full_name sem escape em template de e-mail HTML (ex: sendAccountDeletionEmail).

Por que é explorável: (1) não é explorável hoje, mas é um padrão frágil que pode virar IDOR real num refactor futuro; (2) permite self-XSS de baixo impacto se o nome contiver HTML, explorável apenas em webmails que renderizam HTML/JS.

Evidência:

```
// debts.ts — UPDATE sem AND user_id
UPDATE debts SET ${setClause}, updated_at = NOW() WHERE id = $1 RETURNING *
```

```
// email.ts:36
Ola, ${name}.
```

Sugestão de correção:

1. Adicionar AND user_id = \$N na query UPDATE de debts.ts.
2. Escapar name/full_name (função tipo esc(), já existe padrão similar no frontend) antes de interpolar em templates de e-mail HTML.

Critérios de aceite:

- Query UPDATE de debts.ts filtra por user_id
- Templates de e-mail escapam variáveis de usuário antes da interpolação

--- FIM ISSUE 5 ---